

Лекція

Стандарт Message Passing Interface (MPI)

by Blaise Barney,

Lawrence Livermore National Laboratory

Специфікація інтерфейсу

- Стандарт **Message Passing Interface (MPI)** - це специфікація для розробників та користувачів бібліотек передачі повідомлень. Стандарт MPI не є бібліотекою, це тільки опис того, що має бути в такій бібліотеці.
- MPI відтворює модель паралельного програмування “передачі повідомлень”: паралельні процеси здійснюють обмін повідомленнями (даними) за допомогою комунікаційних операцій.
- Метою стандарту Message Passing Interface є забезпечення достатньо широкого стандарту для написання програм передачі повідомлень. Інтерфейс має бути: практичним, портативним, ефективним, гнучким
- Остання версія MPI: MPI-4.
- Специфікації інтерфейсу визначені у прив’язці до мов програмування C та Fortran90.
- MPI-бібліотеки мов програмування можуть відрізнятись в залежності від версії стандарту MPI, що підтримується. Розробникам/користувачам потрібно це враховувати.

Причини використання MPI

- **Стандартизація** - MPI is the only message passing library which can be considered a standard. It is supported on virtually all HPC platforms. Practically, it has replaced all previous message passing libraries.
- **Портативність** - There is little or no need to modify your source code when you port your application to a different platform that supports (and is compliant with) the MPI standard.
- **Продуктивність** - Vendor implementations should be able to exploit native hardware features to optimize performance. Any implementation is free to develop optimized algorithms.
- **Функціональність** - There are over 430 routines defined in MPI-3, which includes the majority of those in MPI-2 and MPI-1.
- **Доступність** - A variety of implementations are available, both vendor and public domain.

Документація

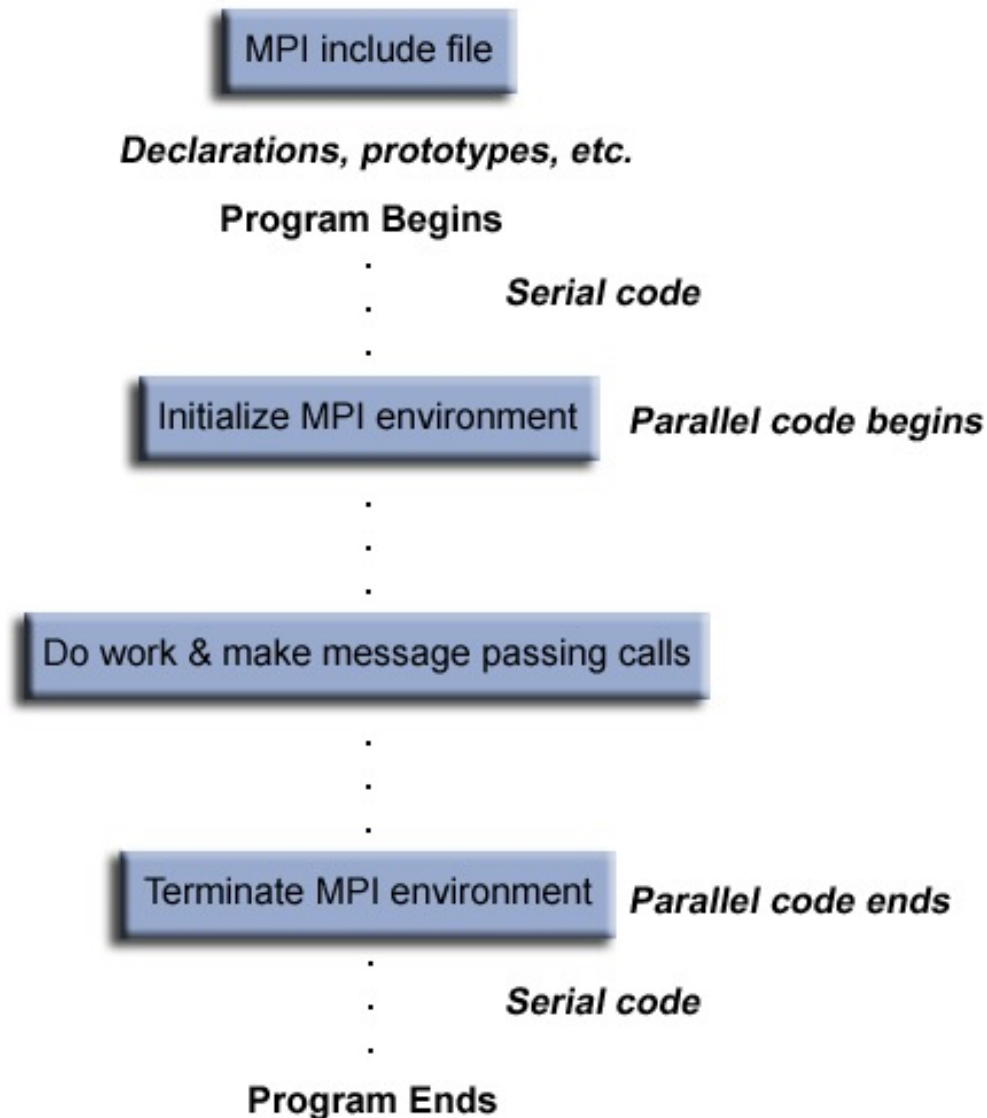
- Документація стандарту MPI (усі версії)
доступна за посиланням:

<http://www.mpi-forum.org/docs/>

Open MPI

- Open MPI – потокобезпечна, відкрита реалізація стандарту MPI мовою C.
- Open MPI доступний для використання на більшості Linux кластерів (LC). Вам потрібно завантажити бажаний пакет інструментів з використанням **use** команди. Наприклад, **use -l (список доступних пакетів) use openmpi-gnu-1.4.3 (використовувати обраний пакет)**
- Це гарантує, що сценарії обгортки MPI LC вказують на потрібну версію Open MPI.
- Більше інформації про Open MPI доступно за посиланням www.open-mpi.org

Загальна структура MPI програми



Комунікатори і групи

- MPI використовує об'єкти комунікатори та об'єкти групи для визначення набору процесів, що можуть спілкуватись один з одним.
- Більшість процедур MPI вимагають, щоб комунікатор був вказаний як аргумент.
- Комунікатори і групи можуть бути задані програмістом. Для спрощення, використовуйте `MPI_COMM_WORLD` для того, щоб вказати на заздалегідь визначений комунікатор, який включає в себе всі ваші процеси MPI.

Rank

- В межах комунікатора кожний процес має свій унікальний цілочисельний ідентифікатор, наданий системою при ініціалізації процесу. Ранг процесу іноді називають "task ID". Нумерація процесів починається з 0.
- Ранг процесу використовується для вказування процесу-відправника чи процесу-отримувача повідомлення. Часто використовується у програмах для контролю виконання обчислень (if rank=0 do this / if rank=1 do that).

Environment Management Routines

- [MPI_Init](#)

Ініціалізує середовище виконання MPI. Ця функція повинна бути викликана у кожній програмі MPI перед викликом будь-яких інших функцій MPI і повинна бути викликана тільки один раз в програмі MPI.

```
MPI_Init      (&argc,&argv)      //      Open      MPI
MPI_INIT (ierr)      // FORTRAN
```

- [MPI_Initialized](#)

Указує, чи був викликаний MPI_Init - повертає логічне значення true (1) або false(0). За стандартом MPI MPI_Init може бути викликаний один і тільки один раз кожним процесом.

```
MPI_Initialized (&flag)
MPI_INITIALIZED (flag,ierr)
```

- [MPI_Finalize](#)

Завершує середовище виконання MPI. Ця функція повинна бути останньою процедурою MPI, яка викликається в кожній програмі MPI - ніякі інші процедури MPI не можуть бути викликані після неї.

```
MPI_Finalize ()
MPI_FINALIZE (ierr)
```

- [MPI_Comm_size](#)

Повертає загальну кількість процесів MPI у вказаному комунікаторі, наприклад MPI_COMM_WORLD. Якщо комунікатор MPI_COMM_WORLD, то він представляє кількість завдань MPI, доступних для вашої програми.

MPI_Comm_size (comm,&size)
MPI_COMM_SIZE (comm,size,ierr)

- [MPI_Comm_rank](#)

Повертає ранг процесу MPI, що викликається, у вказаному комунікаторі. Спочатку кожному процесу буде присвоєно унікальний цілочисельний ранг між 0 і кількістю завдань - 1 в межах комунікатора MPI_COMM_WORLD. Цей ранг часто називають ідентифікатором завдання. Якщо процес стає пов'язаний з іншими комунікаторами, він буде мати унікальний ранг в кожному з них, а також.

MPI_Comm_rank (comm,&rank)
MPI_COMM_RANK (comm,rank,ierr)

- [MPI_Get_processor_name](#)

Повертає ім'я процесора. Також повертає довжину імені. Буфер для "ім'я" має бути розміром не менше MPI_MAX_PROCESSOR_NAME символів. Те, що повертається в "ім'я", залежить від реалізації.

MPI_Get_processor_name (&name,&resultlength)
MPI_GET_PROCESSOR_NAME (name,resultlength,ierr)

- [MPI_Wtime](#)

Повертає час у секундах (подвійна точність) на процесорі, що виконує виклик. Використовується для здійснення замірів часу при вимірюванні часу виконання програми.

MPI_Wtime ()

MPI_WTIME ()

- [MPI_Wtick](#)

Повертає роздільну здатність MPI_Wtime у секундах (подвійна точність).

MPI_Wtick ()

MPI_WTICK ()

- [MPI_Abort](#)

Аварійна зупинка всіх процесів MPI, пов'язаних з комунікатором. У більшості реалізацій MPI зупиняються всі процеси незалежно від вказаного комунікатора.

MPI_Abort (comm,errorcode)

MPI_ABORT (comm,errorcode,ierr)

- [MPI_Get_version](#)

Повертає версію стандарту MPI, реалізованого бібліотекою.

MPI_Get_version (&version,&subversion)

MPI_GET_VERSION (version,subversion,ierr)

Hello, world

```
/** FILE: mpi_hello.c
AUTHOR: Blaise Barney ***/

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#define MASTER 0
int main (int argc, char *argv[]) {
    int numtasks, taskid, len;
    char hostname[MPI_MAX_PROCESSOR_NAME];
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &taskid);
    MPI_Get_processor_name(hostname, &len);
    printf ("Hello from task %d on %s!\n", taskid, hostname);
    if (taskid == MASTER)
        printf("MASTER: Number of MPI tasks is: %d\n", numtasks);
    MPI_Finalize();
}
```

Передача повідомлень один-до-одного

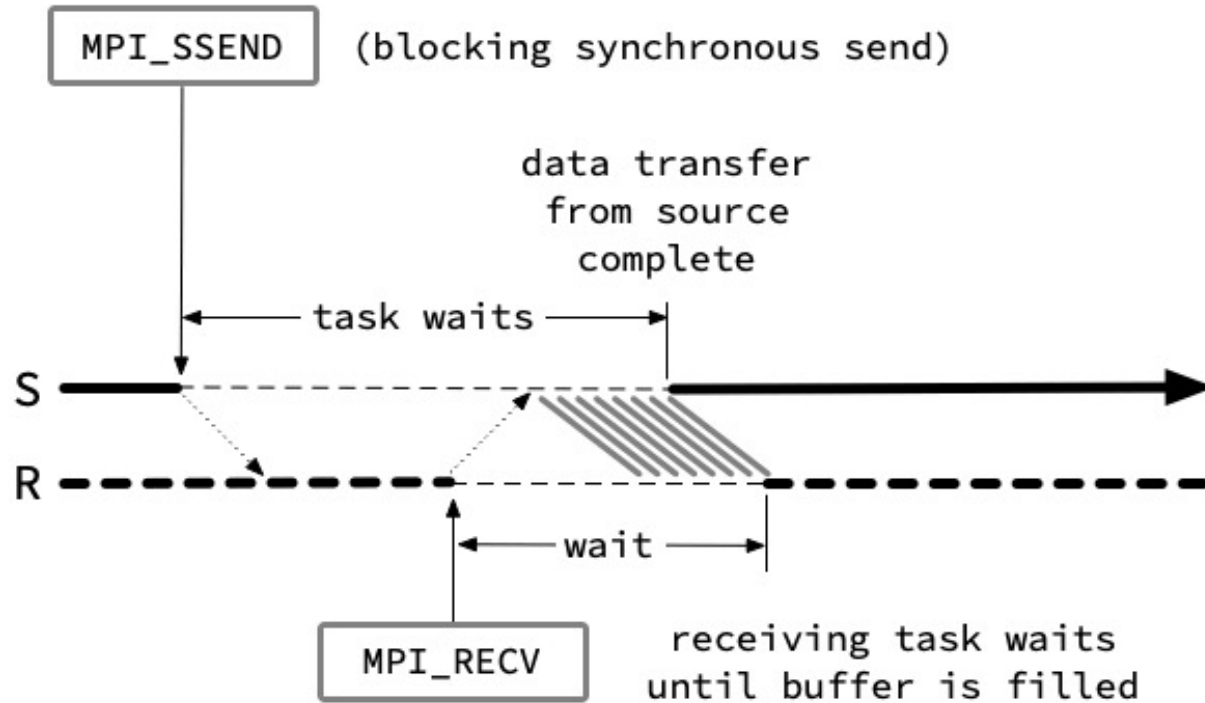
- MPI-методи один-до-одного (point-to-point) реалізують передачу повідомлень між двома, і тільки двома, різними MPI-процесами.
- Існують різні типи передачі (надсилання або отримання), які використовуються для різних цілей. Наприклад:
 - **Blocking send / blocking receive**
 - **Non-blocking send / non-blocking receive**
 - **Buffered send**
 - **Synchronous send**
 - **"Ready" send**
 - **Combined send/receive**
- Будь-який тип надсилання може поєднуватися з будь-яким типом отримання.
- MPI також надає кілька допоміжних методів, пов'язаних з надсилання - отриманням, наприклад, для очікування прибуття повідомлення або моніторингу стану отримувача.

Методи передачі повідомлень один-до-одного

MPI методи передачі повідомлень point-to-point, як правило, мають такий список аргументів:

Blocking sends	<code>MPI_Send(buffer,count,type,dest,tag,comm)</code>
Non-blocking sends	<code>MPI_Isend(buffer,count,type,dest,tag,comm,request)</code>
Blocking receive	<code>MPI_Recv(buffer,count,type,source,tag,comm,status)</code>
Non-blocking receive	<code>MPI_Irecv(buffer,count,type,source,tag,comm,request)</code>

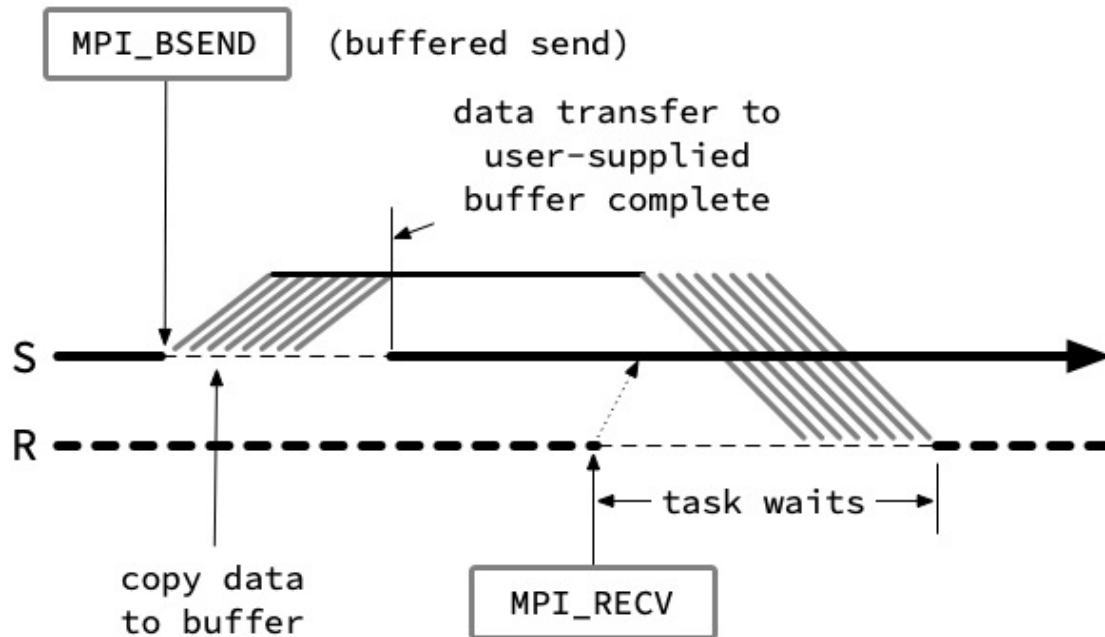
MPI_Send



Cornell Virtual Workshop [<https://cvw.cac.cornell.edu/mpip2p/intro/synchronous-send>]

Синхронізований режим відправки повідомлення означає, що відправка даних розпочнеться тільки після отримання від процесу-отримувача сигналу "ready". Такий сигнал буде отримано процесом-відправником, коли виконання програми на стороні отримувача дійде до відповідної інструкції MPI_Recv.

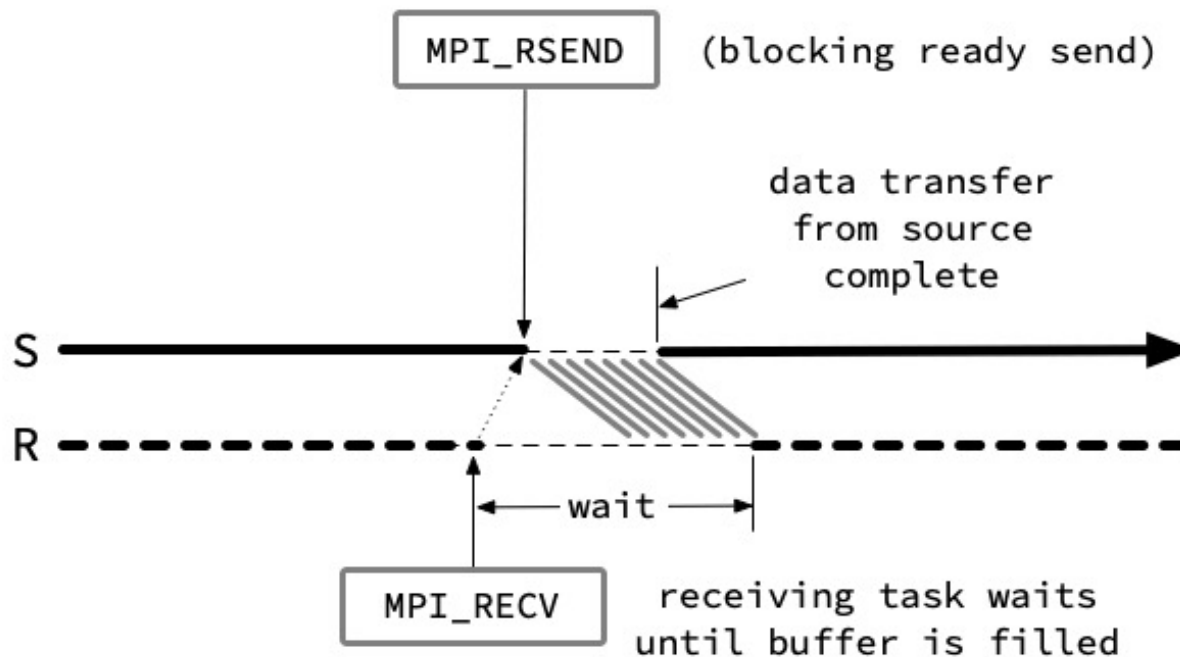
MPI_Bsend



Cornell Virtual Workshop [<https://cvw.cac.cornell.edu/mpip2p/intro/buffered-send>]

Буферизований режим відправки повідомлення означає, що дані будуть скопійовані у буфер тимчасового зберігання даних на стороні процес-відправника, де будуть очікувати на сигнал "ready" від процес-отримувача.

MPI_Rsend

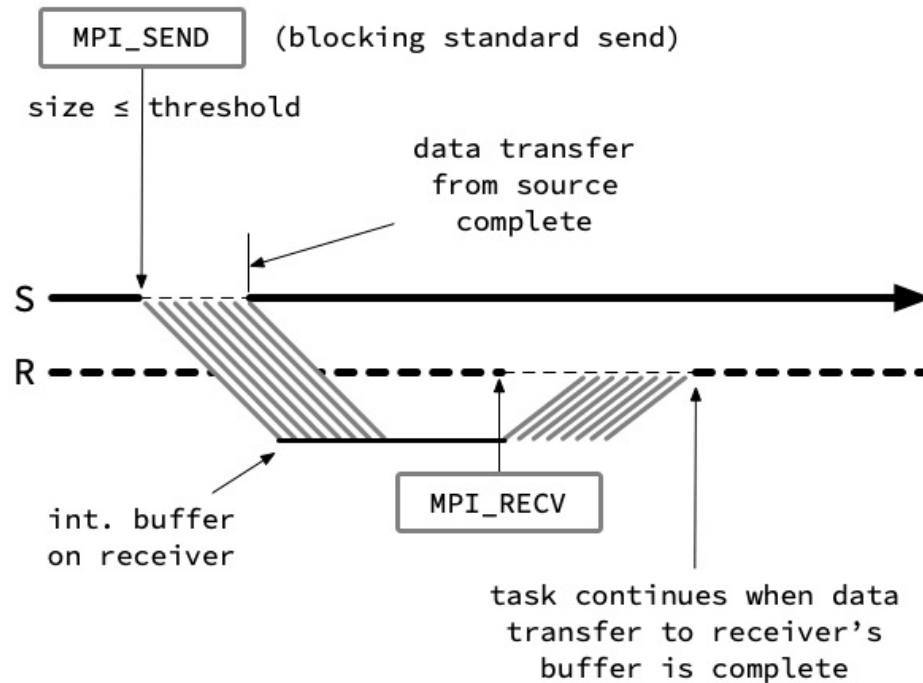


Cornell Virtual Workshop [<https://cvw.cac.cornell.edu/mpip2p/intro/ready-send>]

Режим готовності відправки повідомлення означає, що відправка відбувається у припущенні, що процес-отримувач вже очікує на повідомлення. Проте, якщо процес-отримувач ще не очікує дані, то виникне помилка при пересилці даних. Тобто програміст відповідає за те, що MPI_Recv відбувається до MPI_Send.

Блокування відбувається тільки на час відправки даних в мережу.

MPI_Send



Cornell Virtual Workshop [<https://cvw.cac.cornell.edu/mpip2p/intro/standard-send>]

Стандартний режим відправки повідомлення означає, що передача повідомлення відбувається у буфер тимчасового зберігання даних на стороні процесора-отримувача. Коли процес-отримувач готовий прийняти повідомлення, він читає його зі свого буферу.

Блокування відбувається тільки на час відправки даних у допоміжний буфер.

Переваги використання MPI_Send

- Є безпечним, оскільки не вимагає готовності процесу-отримувача.
- Вимагає блокування процесу обчислень на стороні процесу-відправника тільки на час передачі у буфер процесу-отримувача. На відміну від буферизованої передачі, в момент готовності процесу-отримувача передача даних у буфер процесу-отримувача вже відбулась.
- Реалізація стандартного режиму відправки передбачає певну адаптацію під розмір повідомлення. Для великих повідомлень використовується спосіб близький до синхронізованої передачі повідомлення (з очікуванням сигналу готовності від процесу-отримувача), а для маленьких - до буферизованої (з копіюванням даних у буфер тимчасового зберігання).

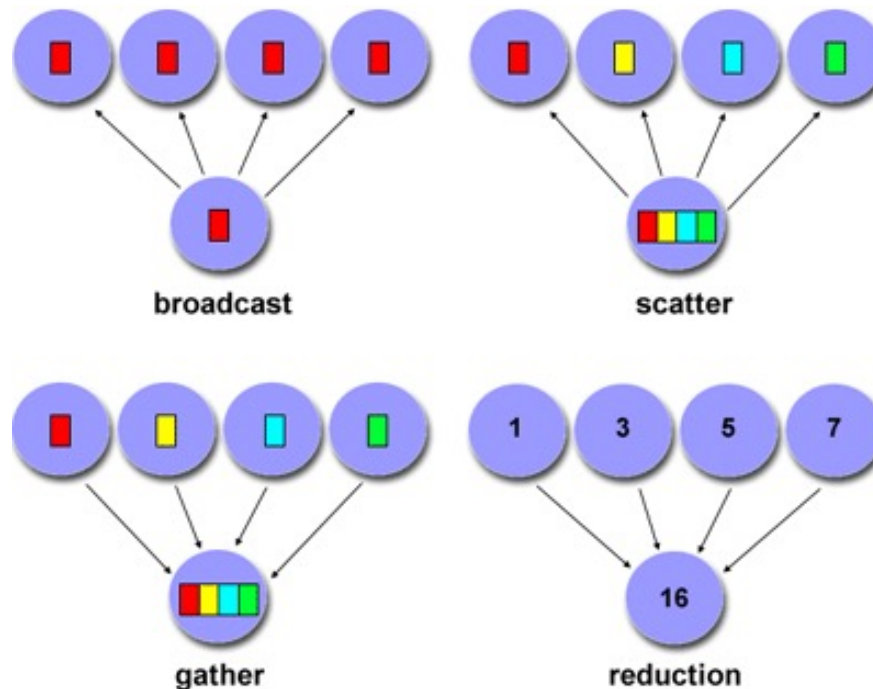
Колективні методи передачі повідомлень

- Колективні комунікаційні процедури залучають до передачі повідомлення **всі** процеси в межах комунікатора.
 - Всі процеси за замовчуванням належать до комунікатора `MPI_COMM_WORLD`.
 - Додаткові комунікатори може визначити програміст.
- Неочікувана поведінка, включно зі збоєм програми, може статися, коли хоча б один процес в комунікаторі не активний.
- Відповідальність програміста полягає в тому, щоб забезпечити, що всі процеси в комунікаторі будуть активні під час виконання колективних операцій.

Колективні методи

Поділяють на такі різновиди:

- **Synchronization** operation - процеси очікують, поки всі учасники групи досягнуть точки синхронізації.
- **Data Movement** operation - broadcast, scatter/gather, all to all.
- **Collective Computation** operation (reductions) - один процес збирає дані від інших процесів і виконує операцію (min, max, add, multiply і т.д.) на цих даних.



Collective Communication Routines

MPI_Barrier

Synchronization operation. Creates a barrier synchronization in a group. Each task, when reaching the MPI_Barrier call, blocks until all tasks in the group reach the same MPI_Barrier call. Then all tasks are free to proceed.

MPI_Barrier (comm)

MPI_BARRIER (comm,ierr)

MPI_Bcast

Data movement operation. Broadcasts (sends) a message from the process with rank "root" to all other processes in the group.

MPI_Bcast (&buffer,count,datatype,root,comm)

MPI_BCAST (buffer,count,datatype,root,comm,ierr)

MPI_Scatter

Data movement operation. Distributes distinct messages from a single source task to each task in the group.

MPI_Scatter (&sendbuf,sendcnt,sendtype,&recvbuf, recvcnt,recvtype,root,comm)

MPI_SCATTER (sendbuf,sendcnt,sendtype,recvbuf, recvcnt,recvtype,root,comm,ierr)

MPI_Gather

Data movement operation. Gathers distinct messages from each task in the group to a single destination task. This routine is the reverse operation of MPI_Scatter.

MPI_Gather (&sendbuf,sendcnt,sendtype,&recvbuf, recvcount,recvtype,root,comm)

MPI_GATHER (sendbuf,sendcnt,sendtype,recvbuf, recvcount,recvtype,root,comm,ierr)

MPI_Allgather

Data movement operation. Concatenation of data to all tasks in a group. Each task in the group, in effect, performs a one-to-all broadcasting operation within the group.

MPI_Allgather (&sendbuf,sendcount,sendtype,&recvbuf, recvcount,recvtype,comm)

MPI_ALLGATHER (sendbuf,sendcount,sendtype,recvbuf, recvcount,recvtype,comm,info)

MPI_Reduce

Collective computation operation. Applies a reduction operation on all tasks in the group and places the result in one task.

MPI_Reduce (&sendbuf,&recvbuf,count,datatype,op,root,comm)

MPI_REDUCE (sendbuf,recvbuf,count,datatype,op,root,comm,ierr)